

Jacobian lens,輕量版

Anthropic 的 Jacobian lens 主張語言模型的中間層裡有一個 workspace,放著還沒說出口的想法,可以讀出來也可以改。這份 notebook 用 MLX 在 Apple Silicon 上把這把尺怎麼算出來 講清楚,再用一個例子看它讀到什麼。

主線:

1. 準備:load model、四個貫穿全篇的工具函式
2. logit lens:借最後那顆頭讀中間層,以及它讀不到的東西
3. 算出 J :先用兩次 forward 拿到 Jh ,再用 VJP 拿矩陣本體,最後換成公開 lens
4. 範例:法國的首都
5. 小結

附錄是論文那些實驗,用主線建好的工具跑,可以跳著看:換個語言問、J-space 本體、把 France 換成 China、三個 workspace 面向。完整版 [jacobian space from scratch qwen36-27b.ipynb](#) ([jacobian space from scratch qwen36-27b.ipynb](#)),有全部的實驗、對照組和數字。

跑之前要有 models/Qwen3.6-27B-4bit (權重)和同一個資料夾裡的 jlens.npz (公開 lens),兩個都照 [README \(README.md\)](#) 的步驟下載。這份存的時候沒有留輸出,demo 前先 Run All 一次;最慢的一格是 S3.2,那裡要跑 8 次反傳。

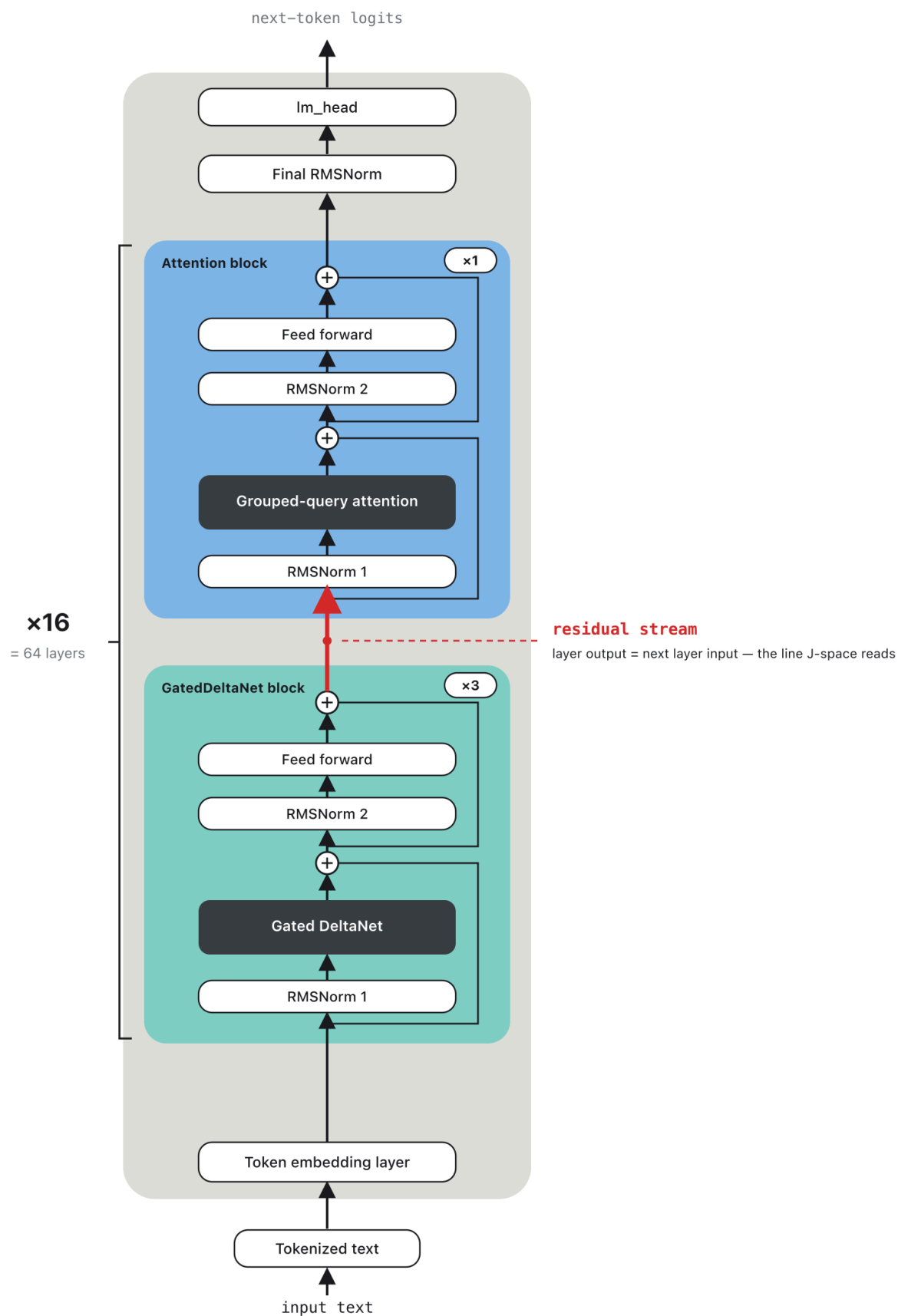
Source:論文 (<https://transformer-circuits.pub/2026/workspace/index.html>) · 官方 code (<https://github.com/anthropics/jacobian-lens>) · Neuronpedia demo (<https://www.neuronpedia.org/qwen3.6-27b/jlens>).

1. 準備

Qwen3.6-27B 是 64 層 decoder,每一層都在同一條 residual stream 上加東西:

```
h <- h + mixer(RMSNorm(h))  
h <- h + FFN(RMSNorm(h))
```

mixer 有兩種,每 4 層一組:3 層 GatedDeltaNet(固定大小的 state,線性 attention)配 1 層 full attention。兩者怎麼取用前文對後面的 lens 沒有影響,重要的只有一件事:64 層的輸出都停在同一個座標系裡,所以同一把尺可以量每一層 ([A Mathematical Framework for Transformer Circuits](https://transformer-circuits.pub/2021/framework/index.html) (<https://transformer-circuits.pub/2021/framework/index.html>))。



先修 mlx-llm 的一個 bug,再 load。這一格跟方法無關,純粹是環境問題。

```
In [ ]: def patch_qwen3_5():
    """mlx-vlm #1548: 0.6.4 misses the +1.0 shift on qwen3_5 RMSNorm weights,
    which garbles every output. Must run before load()."""
    import mlx_vlm.models.qwen3_5.qwen3_5 as _q
    keys = (".input_layernorm.weight", ".post_attention_layernorm.weight",
            "model.norm.weight", ".q_norm.weight", ".k_norm.weight")
    if getattr(_q.Model.sanitize, "_patched", False):
        return
    base = _q.sanitize_key

    def sanitize(self, weights):
        shift = any("mtp." in k for k in weights) or any(
            "conv1d.weight" in k and v.shape[-1] != 1 for k, v in weights.items())
        weights = {k: v for k, v in weights.items() if "mtp." not in k}
        if self.config.text_config.tie_word_embeddings:
            weights.pop("lm_head.weight", None)
        out = {}
        for k, v in weights.items():
            k = base(k)
            if "conv1d.weight" in k and v.shape[-1] != 1:
                v = v.moveaxis(2, 1)
            if shift and any(k.endswith(s) for s in keys) and v.ndim == 1:
                v = v + 1.0
            out[k] = v
        return out

    sanitize._patched = True
    _q.Model.sanitize = sanitize

patch_qwen3_5()
```

```
In [ ]: import mlx.core as mx
from mlx_vlm import load

model, processor = load("models/Qwen3.6-27B-4bit")
tokenizer = processor.tokenizer

llm = model.language_model      # the decoder + lm_head
inner = llm.model               # embed_tokens, the 64 layers, the final norm
cfg = model.config.text_config

DTYPE = inner.norm.weight.dtype # bfloat16: everything fed back into the model must match
D_MODEL = cfg.hidden_size
N_LAYERS = len(inner.layers)

print(f"{N_LAYERS} layers, d_model {D_MODEL}, vocab {cfg.vocab_size}, activations {DTYPE}")
for i, layer in enumerate(inner.layers[:4]):
    print(f"  layer {i}: " + ("GatedDeltaNet" if layer.is_linear else "full attention"))
print(f"  ... same 4-layer pattern x {N_LAYERS // 4}")
```

四個工具函式

整份 notebook 的計算都走這四個函式:

- forward 一層一層跑,每層之後呼叫 hook 。所有實驗(抽 residual、注入、介入)都是換一個 hook ,不再各寫一遍 forward。
- residuals 收集每一層的輸出。
- readout 把模型自己最後那顆頭(final RMSNorm + lm_head)套到任何一條 residual 上。
- final_scores 跑一次(可以帶 hook 的)forward,回傳最後一個位置的 token 分數。

手動一層層跑就得自己準備兩種 mask 和 MRoPE 位置,才跟官方 forward 一致。

```

In [ ]: from mlx_vlm.models.qwen3_5.language import (
        _create_qwen3_5_attention_mask, _create_qwen3_5_ssm_mask)

def forward(ids, hook=None, stop=None):
    """Run the decoder layer by layer, returning the residual after layer `stop`.

    hook(h, layer_index) -> h runs after each layer; that is where every
    experiment below plugs in.
    """
    h = inner.embed_tokens(ids)
    full_mask = _create_qwen3_5_attention_mask(h, None)
    ssm_mask = _create_qwen3_5_ssm_mask(h, None)
    # MRoPE has 3 axes (time, height, width) for images; text-only input puts the
    # same token index on all three.
    pos = mx.tile(mx.arange(h.shape[1])[None, None, :], (3, 1, 1))

    stop = N_LAYERS - 1 if stop is None else stop
    for l, layer in enumerate(inner.layers):
        h = layer(h, mask=(ssm_mask if layer.is_linear else full_mask),
                    cache=None, position_ids=pos, position_embeddings=None)
        if hook is not None:
            h = hook(h, l)
        if l == stop:
            break
    return h

def residuals(ids):
    """residuals(ids)[l] is the residual stream after layer l."""
    out = []

    def collect(h, l):
        out.append(h)
        return h

    forward(ids, hook=collect)
    return out

def readout(h):
    """Score every token with the model's own final head: lm_head(RMSNorm(h)).

    Takes a single residual vector or a whole (batch, position, d_model) array.
    """
    x = h.astype(DTYPE).reshape(-1, D_MODEL)
    scores = llm.lm_head(inner.norm(x)).astype(mx.float32)
    return scores.reshape(*h.shape[:-1], -1)

def final_scores(ids, hook=None):
    """Token scores at the last position, after a possibly edited forward pass."""
    return readout(forward(ids, hook=hook)[0, ids.shape[1] - 1])

```

```
In [ ]: def encode(text):
        return mx.array(tokenizer.encode(text))[None]

def tid(word):
    """First token id of a spelling, e.g. tid(" Paris")."""
    return tokenizer.encode(word)[0]

def where(ids, word):
    """Position of the first token whose text contains `word`."""
    toks = [tokenizer.decode([int(t)]) for t in ids[0].tolist()]
    return next(i for i, t in enumerate(toks) if word in t)

def prob(scores, token):
    """Probability of one token, given as an id or as a string to tokenize."""
    return float(mx.softmax(scores)[token if isinstance(token, int) else tid(token)])

def top_tokens(scores, k=5):
    return [tokenizer.decode([int(t)]) for t in mx.argsort(-scores)[:k].tolist()]

def top1(scores):
    return tokenizer.decode([int(mx.argmax(scores))])

PROMPT = "The capital of France is" # the one example, used from here to the end

ids = encode(PROMPT)
print("tokens      :", [tokenizer.decode([int(t)]) for t in ids[0].tolist()])
print("next token  :", repr(top1(final_scores(ids))))
```

畫圖的設定。灰色是對照或基準(logit lens、baseline、原本的答案),藍色是 Jacobian lens 這一側,橘色是被介入之後長出來的東西。圖上的字一律用英文,matplotlib 預設字型沒有中文。

```
In [ ]: import matplotlib.pyplot as plt

INK, LOGIT, JLEN, SWAPPED = "#14171a", "#8a9094", "#2f6f8f", "#c1662f"

plt.rcParams.update({
    "figure.dpi": 130, "figure.facecolor": "white", "savefig.bbox": "tight",
    "font.size": 8.5, "axes.titlesize": 9,
    "axes.spines.top": False, "axes.spines.right": False,
    "axes.edgecolor": "#c8cccd", "grid.color": "#e8e8ec",
    "legend.frameon": False, "legend.fontsize": 8,
    "lines.linewidth": 1.6, "lines.markersize": 4,
})

def panel(ax, title="", xlabel="", ylabel="", ymax=None, axis="y"):
    """One consistent look for every panel: title on the left, one grid axis."""
    ax.set_title(title, loc="left", color=INK)
    ax.set_xlabel(xlabel)
    ax.set_ylabel(ylabel)
    ax.grid(axis=axis)
    ax.set_axisbelow(True)
    if ymax is not None:
        ax.set_ylim(0, ymax)
    return ax

def grouped_bars(ax, labels, series, colors):
    """series = {name: [one value per label]}, drawn as one group of bars per label."""
    width = 0.8 / len(series)
    for i, ((name, vals), color) in enumerate(zip(series.items(), colors)):
        offset = (i - (len(series) - 1) / 2) * width
        ax.bar([x + offset for x in range(len(labels))], vals,
                width=width, color=color, label=name)
    ax.set_xticks(range(len(labels)))
    ax.set_xticklabels(labels)

print("plot style ready")
```

2. Logit lens

可解釋性要問的是模型吐出答案的路上,中間發生了什麼。既然每一層的輸出都在同一個座標系上,就可以拿同一把尺去量每一層。logit lens (nostalgebraist 2020 (<https://www.lesswrong.com/posts/AcKRB8wDpdaN6v6ru/interpreting-gpt-the-logit-lens>)) 把模型最後那顆頭借來,本來只讀最終的 h_L ,現在對每一層的 h_ℓ 都讀一次:

$$\text{LogitLens}(h_\ell) = W_U \cdot \text{norm}(h_\ell)$$

norm 是 final RMSNorm, W_U 是 lm_head 。這就是上面的 readout ,對每一層呼叫一次而已。

```
In [ ]: res = residuals(ids)
last = ids.shape[1] - 1

answer = int(mx.argmax(readout(res[-1][0, last])))
ANS = tokenizer.decode([answer])

logit_p, tops = [], []
for l in range(N_LAYERS):
    scores = readout(res[l][0, last])
    logit_p.append(prob(scores, answer))
    tops.append(top1(scores))
first_top1 = next(l for l in range(N_LAYERS) if tops[l] == ANS)

fig, ax = plt.subplots(figsize=(5.6, 2.5))
panel(ax, f"logit lens: p({ANS.strip()}) at the last position", "layer",
      "p(answer)", 1.0)
ax.plot(range(N_LAYERS), logit_p, color=LOGIT)
ax.axvline(first_top1, color=LOGIT, lw=0.9, ls=":")
ax.text(first_top1 - 1, 0.9, f"top-1 becomes {ANS!r} at L{first_top1}",
      color=LOGIT, ha="right", fontsize=8)
plt.show()

print(f"{PROMPT!r} -> {ANS!r}")
print("p(answer) every 8th layer:")
print(" " + " ".join(f"L{l}:{logit_p[l]:.3f}" for l in range(0, N_LAYERS, 8)))
print("\nmid-stack top-1: " + " ".join(f"L{l}:{tops[l]!r}" for l in (20, 30, 40, 50)))
```

前面幾十層讀不到答案,top-1 是各種碎片, $p(\text{answer})$ 是 0.000,要到 50 幾層才起來。

讀不到不代表模型內部沒有答案,也可能是這把尺不夠準。把 h_ℓ 丟進最後那顆頭,隱含的 假設是從第 ℓ 層到終點的那幾十層什麼都不做:

$$\text{LogitLens}(h_\ell) = W_U \cdot \text{norm}(h_\ell) \iff \frac{\partial h_L}{\partial h_\ell} \approx I$$

這個假設對最後幾層還堪用,對中層太粗。答案可能早就算出來了,只是還沒轉到最後那顆頭 讀得到的方向上。

3. 算出 J

Anthropic 的做法是把上面假設成 I 的那一項算出來:

$$J_\ell = \frac{\partial h_L}{\partial h_\ell} \quad \text{JLens}(h_\ell) = W_U \cdot \text{norm}(J_\ell h_\ell)$$

J_ℓ 是 5120×5120 的矩陣,描述第 ℓ 層的 residual 動一點、最終的 residual 會怎麼跟著動。乘上它等於把 h_ℓ 搬到最終 residual 附近再用同一顆頭讀,跟 logit lens 的差別只有這一步。它是在某個輸入、某個位置附近展開的一階近似,不是全域的座標轉換。

這一節做三件事:先不碰微分套件拿到 Jh (S3.1),再把矩陣的數值算出來(S3.2),最後說明為什麼實務上要換成公開的 lens(S3.3)。

3.1 先不談微分:推一下,看終點怎麼動

J_ℓ 的定義就是一階展開

$$h_L(h_\ell + d) \approx h_L(h_\ell) + J_\ell d$$

所以想要 $J_\ell h_\ell$,不必先把矩陣算出來再乘:把 $d = \varepsilon h_\ell$ 推進去,跑到最後,看最終的 residual 動了多少,除掉 ε 就是了。兩次 forward,沒有微分套件,沒有矩陣。

因果的關係要注意:第 ℓ 層某個位置動一下,終點是那個位置以後全部都會動,而 lens 用到的只有同一個位置進、同一個位置出那一塊。所以下面進出都取同一個 pos。

```
In [ ]: def jacobian_times(ids, direction, src, pos, tgt=None, eps=0.25):
        """J @ direction at position `pos`, without ever building J.

        J is defined by  $h_{tgt}(h_{src} + d) = h_{tgt}(h_{src}) + J d + O(d^2)$ , so nudge the
        residual at (layer src, position pos) by  $eps * direction$ , see how far the
        final residual at that same position moves, and divide by  $eps$ .

        eps trades off two errors: the model runs in bfloat16 (~3 significant
        digits), so too small and the movement is lost to rounding; too large and
        the 10-ish layers above are no longer close to linear. 0.25 sits between.
        """
        tgt = N_LAYERS - 1 if tgt is None else tgt
        step = (eps * direction).astype(DTYPE)

        def nudge(h, l):
            if l == src:
                h[0, pos] = h[0, pos] + step
            return h

        base = forward(ids, stop=tgt)[0, pos]
        moved = forward(ids, hook=nudge, stop=tgt)[0, pos]
        return ((moved - base) / eps).astype(mx.float32)
```

要讀的位置得挑一下。公開 lens(S3.3)擬合時跳過每條 prompt 的前 16 個 position ([issue #5 \(https://github.com/anthropics/jacobian-lens/issues/5\)](https://github.com/anthropics/jacobian-lens/issues/5)),而 The capital of France is 只有 5 個 token,讀的位置落在它沒擬合過的區間。所以在前面墊一段無關的話,把要讀的位置推到 16 以後 —— 問題還是同一個問題,S4 會把墊過和沒墊過的擺在一起看。

```
In [ ]: FILLER = ("Here are some notes about geography that provide background context "
                "for the question that follows this introductory sentence. ")
PADDED = FILLER + PROMPT
SRC = 54 # a mid-to-late layer, where the two lenses differ most

demo_ids = encode(PADDED)
POS = demo_ids.shape[1] - 1 # the position being read, and it is past 16
h54 = residuals(demo_ids)[SRC][0, POS].astype(mx.float32)
Jh54 = jacobian_times(demo_ids, h54, SRC, POS)

print(f"{PADDED!r}")
print(f" {demo_ids.shape[1]} tokens, reading position {POS}, layer {SRC}\n")
print(f" ||h|| = {float(mx.linalg.norm(h54)):8.1f}")
print(f" ||J h|| = {float(mx.linalg.norm(Jh54)):8.1f}")
cos = float(h54 @ Jh54 / (mx.linalg.norm(h54) * mx.linalg.norm(Jh54)))
print(f" cos(h, J h) = {cos:+.3f} (J = I would give +1.000)")
print()
print("read the same residual two ways:")
print(f" logit lens {top_tokens(readout(h54))}")
print(f" jacobian lens {top_tokens(readout(Jh54))}")
```

$J_\ell h_\ell$ 跟 h_ℓ 不同向,兩把尺讀出來的字也不一樣 —— 差別完全來自這一步,同一條 residual、同一顆讀出頭。

3.2 矩陣本體:一列一次 VJP

要看 J_ℓ 的數值就得真的微分。 J 的第 i 列是「終點第 i 個座標」對整條 h_ℓ 的梯度,那正好是一次 vector-Jacobian product:在輸出端只放一個 1 在 (pos, i) ,反傳,看落在注入的 d 上的梯度是多少。

注入的 d 是全零,加上去不改變任何東西,要的只是梯度會流過它。

MLX 這邊有兩個實務細節,寫在 differentiable_linear_attn 裡:GatedDeltaNet 在 eval 模式走的是沒有定義梯度的 metal kernel,要切到 train 模式(同一條遞迴,但用純 ops 寫); 那條路內部會呼叫 mx.async_eval,在 mx.vjp 裡不合法,得暫時關掉。

```
In [ ]: import contextlib

@contextlib.contextmanager
def differentiable_linear_attn():
    """Make the GatedDeltaNet layers differentiable for the duration.

    1. eval mode runs a fused metal kernel with no gradient defined; train mode
       runs the same recurrence as plain ops, which mx.vjp can traverse.
    2. that path calls mx.async_eval internally, which is illegal inside mx.vjp.
    """
    saved_async_eval = mx.async_eval
    mx.async_eval = lambda *a, **kw: None
    for layer in inner.layers:
        if layer.is_linear:
            layer.linear_attn.train(True)
    try:
        yield
    finally:
        mx.async_eval = saved_async_eval
        for layer in inner.layers:
            if layer.is_linear:
                layer.linear_attn.train(False)

def jacobian_row(ids, row, src, pos, tgt=None):
    """Row `row` of  $J = d h_{tgt}[pos] / d h_{src}[pos]$ , exactly, with one VJP."""
    tgt = N_LAYERS - 1 if tgt is None else tgt
    seq = ids.shape[1]

    delta = mx.zeros((1, seq, D_MODEL), dtype=DTYPE) # adding 0 changes the forward pass
    seed = mx.zeros((1, seq, D_MODEL), dtype=DTYPE) # not at all; we only want its gradient
    seed[0, pos, row] = 1.0

    def add_delta(d):
        return forward(ids, hook=lambda h, l: h + d if l == src else h, stop=tgt)

    with differentiable_linear_attn():
        _, (grad,) = mx.vjp(add_delta, [delta], [seed])
        mx.eval(grad)
    return grad[0, pos].astype(mx.float32)
```

```
In [ ]: import time

N_ROWS = 8 # one VJP per row, and the full matrix is 5120 rows for one layer

t0 = time.perf_counter()
rows = mx.stack([jacobian_row(demo_ids, r, SRC, POS) for r in range(N_ROWS)])
print(f"{N_ROWS} rows of J[L{SRC}] in {time.perf_counter() - t0:.0f}s\n")

print(f"J[0,0] = {float(rows[0, 0]):.3f} (the logit lens assumes 1.000)")
print(f"||J[0]|| = {float(mx.linalg.norm(rows[0])):.2f} most of the row sits off the diagonal")
print()
print("first 8 entries of J h, computed two independent ways:")
print(" vjp rows @ h " + " ".join(f"{float(v):8.2f}" for v in rows @ h54))
print(" $3.1 nudge " + " ".join(f"{float(v):8.2f}" for v in Jh54[:N_ROWS]))
```

兩條路算出同一組數字,一邊是反傳,一邊只是推一下看結果,誤差來自 bfloat16 和一階近似。

$J[0,0]$ 不是 1,而整列的長度比對角元素大不少,也就是說對角線之外還有很多東西, logit lens 把它們全部丟掉了。

3.3 換成公開的 lens

上面是一條 prompt、一個位置算出來的。要當成尺來用還差兩件事:整個 5120×5120 矩陣(每層 5120 次 VJP,而且 64 層都要),以及不綁特定輸入。Anthropic 的做法是對很多條 prompt 擬合一個 J_{ℓ} 。這裡載 Neuronpedia 放出來的結果 ([neuronpedia/jacobian-lens](https://huggingface.co/neuronpedia/jacobian-lens)) (<https://huggingface.co/neuronpedia/jacobian-lens>), 跟剛剛自己算的那 8 列比一下方向。

```
In [ ]: _lens = mx.load("models/Qwen3.6-27B-4bit/jlens.npz")
src_layers = [int(x) for x in _lens["__source_layers__"].tolist()]
n_prompts = int(_lens["__n_prompts__"].tolist()[0])
Jlens = {l: _lens[f"J_{l}"] for l in src_layers} # {layer: (5120, 5120) fp16}

def transport(h, l):
    """Move a layer-l residual into the final-residual basis: J_l @ h."""
    J = Jlens[l]
    return (h.astype(J.dtype) @ J.T).astype(DTYPE)

ours = rows.reshape(-1)
public = Jlens[SRC].astype(mx.float32)[:N_ROWS].reshape(-1)
cos = float(ours @ public / (mx.linalg.norm(ours) * mx.linalg.norm(public)))

print(f"public lens: layers {src_layers[0]}..{src_layers[-1]} "
      f"({len(src_layers)} of {N_LAYERS}), fitted on {n_prompts} prompts")
print(f"cos(our {N_ROWS} rows from 1 prompt, the same rows of the public lens) = {cos:.3f}")
```

一條 prompt 算出來的方向跟上千條擬合的結果大致同向,不必期待貼合:一個是這條輸入在這個位置的局部微分,一個是平均。後面統一用公開 lens,這樣每一層都有、而且不會每換一條 prompt 就得重算。

代價是這把尺帶了跨 prompt 的先驗進來,不再只反映當前這條輸入,這是小結那幾點懷疑的第一條。

4. 範例:法國的首都

兩把尺並排:同一次 forward、同一條 residual、同一類讀出頭,唯一的差別是有沒有乘 J_ℓ 。左邊是原本那句 5 個 token 的問題(讀的位置在 lens 沒擬合過的區間),右邊是 S3.1 墊過的版本。

```
In [ ]: LS = sorted(l for l in Jlens if l >= 44)

def lens_curves(prompt):
    """p(answer) at the last position, layer by layer, for both lenses."""
    ids = encode(prompt)
    res = residuals(ids)
    last = ids.shape[1] - 1
    ans = int(mx.argmax(readout(res[-1][0, last])))
    logit = [prob(readout(res[l][0, last]), ans) for l in LS]
    jac = [prob(readout(transport(res[l][0, last], l)), ans) for l in LS]
    return last, ans, logit, jac

CASES = [("short prompt, outside the fitted range", PROMPT),
         ("padded prompt, inside the fitted range", PADDED)]

fig, axes = plt.subplots(1, 2, figsize=(7.6, 2.8), sharey=True)
out = []
for ax, (title, prompt) in zip(axes, CASES):
    pos, ans, logit, jac = lens_curves(prompt)
    out.append((title, pos, ans, logit, jac))
    ax.plot(LS, logit, color=LOGIT, marker="o", ms=3, label="logit lens")
    ax.plot(LS, jac, color=JLENS, marker="o", ms=3, label="Jacobian lens")
    panel(ax, f"{title}\nreading position {pos}", "layer", "", 1.0)
axes[0].set_ylabel("p(answer)")
axes[1].legend(loc="lower right")
plt.show()

for title, pos, ans, logit, jac in out:
    print(f"{title} answer {tokenizer.decode([ans])!r}, position {pos}")
    for l, a, b in zip(LS, logit, jac):
        if l % 2 == 0:
            print(f" L{l} logit {a:.3f} jacobian {b:.3f}")
```

Jacobian lens 的曲線早好幾層起來。墊過的那條落差更大,所以短 prompt 那版是低估,不是製造出來的。

再看同一件事的另一個切面:每一層兩把尺各自的 top-3。

```
In [ ]: def lens_trace(prompt, layers, k=3):
        """Both lenses' top-k at the last position, layer by layer."""
        ids = encode(prompt)
        res = residuals(ids)
        last = ids.shape[1] - 1
        print(f"{prompt!r}\n output top-1 = {top1(readout(res[-1][0, last]))!r}\n")
        print(f" {'':7s}{'jacobian top-3':40s}logit top-3")
        for l in layers:
            h = res[l][0, last]
            jac = " ".join(repr(t) for t in top_tokens(readout(transport(h, l)), k))
            log = " ".join(repr(t) for t in top_tokens(readout(h), k))
            print(f" L{l:<6d}{jac:40s}{log}")

lens_trace(PADDED, [l for l in LS if l % 3 == 0])
```

Jacobian lens 那一欄先出現答案,logit lens 那一欄還在碎片和標點上。這就是整個方法的主張:答案早就在 residual 裡了,只是還沒轉到最後那顆頭讀得到的方向上,乘上 J_ℓ 就是把它轉過去。

一個必要的自我提醒: Paris 不在 prompt 裡,所以讀到它不能用「輸入裡本來就有這個字」解釋。要探測的字本來就在 prompt 裡的話,lens 會在那個位置直接把它讀成第一名 ([issue #5](https://github.com/anthropics/jacobian-lens/issues/5) (<https://github.com/anthropics/jacobian-lens/issues/5>)),那種結果沒有意義。

5. 小結

- residual stream 是所有層共用的座標系,所以才有得讀。
- logit lens 把最後那顆頭借來讀中間層,便宜,但暗中假設 $J_\ell = I$ 。
- J_ℓ 不用微分套件也能碰:推一下 residual、看終點怎麼動,除掉 ϵ 就是 $J_\ell h_\ell$,兩次 forward。要矩陣的數值就一列一次 `mx.vjp`,兩條路算出同一組數字(\$3.1、\$3.2)。
- 實務上用跨 prompt 擬合好的公開 lens,才有全部 64 層、也不綁單一輸入(\$3.3)。
- 乘上 J_ℓ 之後,同一條 residual、同一顆頭,答案早好幾層讀到,而且讀的位置要落在 lens 擬合過的區間才算數(\$4)。

該保留的懷疑

- 線性近似。 J_ℓ 是一階展開,後面幾十層並不是線性的。
- 全域先驗。公開 lens 是跨上千條 prompt 平均的,會帶進跟前輸入無關的偏好。
- **input-copying**。要探測的字本來就在 prompt 裡的話,讀到它反映的是輸入而不是內部想法 ([issue #5](https://github.com/anthropics/jacobian-lens/issues/5) (<https://github.com/anthropics/jacobian-lens/issues/5>))。
- 名字取得不好。J-space 不是線性子空間,照論文自己的 methods,它是稀疏限制下的一堆多面錐的聯集(附錄 B 量過它佔多少)。
- 循環論證的疑慮。一個依「對最終輸出的影響」定義出來的空間,本來就會跟最終輸出高度相關。
- 不是每個實驗都能重現 (Nanda 團隊的 review (<https://www.lesswrong.com/posts/zFJ3ZdQwrTWE9jT5S/a-review-of-anthropic-s-global-workspace-paper>)、[tao-hpu/jspace-replication](https://github.com/tao-hpu/jspace-replication) (<https://github.com/tao-hpu/jspace-replication>))。

參考資料

- [Verbalizable Representations Form a Global Workspace in Language Models](https://transformer-circuits.pub/2026/workspace/index.html) (<https://transformer-circuits.pub/2026/workspace/index.html>) 論文
- [anthropics/jacobian-lens](https://github.com/anthropics/jacobian-lens) (<https://github.com/anthropics/jacobian-lens>) 官方實作
- [neuronpedia/jacobian-lens](https://huggingface.co/neuronpedia/jacobian-lens) (<https://huggingface.co/neuronpedia/jacobian-lens>) 擬合好的 lens
- [Interpreting GPT: the logit lens](https://www.lesswrong.com/posts/AcKRB8wDpdaN6v6ru/interpreting-gpt-the-logit-lens) (<https://www.lesswrong.com/posts/AcKRB8wDpdaN6v6ru/interpreting-gpt-the-logit-lens>)
- [A Mathematical Framework for Transformer Circuits](https://transformer-circuits.pub/2021/framework/index.html) (<https://transformer-circuits.pub/2021/framework/index.html>)
- 完整版 notebook: [jacobian_space_from_scratch_qwen36-27b.ipynb](https://github.com/jacobian_space_from_scratch_qwen36-27b.ipynb) ([jacobian_space_from_scratch_qwen36-27b.ipynb](https://github.com/jacobian_space_from_scratch_qwen36-27b.ipynb))

附錄:論文的實驗

主線到這裡結束。以下把論文的幾個實驗搬過來,用的都是上面那些工具函式。四個附錄各自 獨立,但要照順序跑(B 定義的 v_l 是 C 和 D 的前提)。

	在做什麼
A	換個語言問同一件事
B	J-space 本體:稀疏非負分解
C	介入:把 France 換成 China ,四個任務一起改
D	三個 workspace 面向:定向調節、概念消除、內部推理

每一節的解讀會引用完整版跑出來的數字,那份有全部五個面向和更多對照組。

附錄 A:換個語言問同一件事

同一個問題用四個語言問,看 Paris 這個概念(好幾種拼法)最早在哪一層進 top-5。

```
In [ ]: # ' Paris', 'Paris', and the Chinese '巴黎'
PARIS = {11751, 57590, 109705}

def first_top5(prompt, targets, k=5):
    """First layer where any target token enters the top-5, for each lens."""
    ids = encode(prompt)
    res = residuals(ids)
    last = ids.shape[1] - 1
    hit = {"jacobian": None, "logit": None}
    for l in sorted(Jlens):
        h = res[l][0, last]
        for name, scores in (("jacobian", readout(transport(h, l))),
                             ("logit", readout(h))):
            if hit[name] is None and targets & set(
                int(t) for t in mx.argsort(-scores)[:k].tolist()):
                hit[name] = l
    return hit["jacobian"], hit["logit"]

PROMPTS = [("English", PROMPT),
            ("Chinese", "法国的首都"),
            ("French", "La capitale de la France est"),
            ("Japanese", "フランスの首都は")]

rows_lang = [(lang, p) + first_top5(p, PARIS) for lang, p in PROMPTS]

fig, ax = plt.subplots(figsize=(5.6, 2.4))
for i, (lang, _, fj, fl) in enumerate(rows_lang):
    y = len(rows_lang) - 1 - i
    if fj is not None and fl is not None:
        ax.plot([fj, fl], [y, y], color="#c8cccd", lw=1.4, zorder=1)
    for hit, color, name in ((fj, JLENS, "Jacobian lens"), (fl, LOGIT, "logit lens")):
        if hit is not None:
            ax.scatter([hit], [y], color=color, s=26, zorder=2,
                       label=name if i == 0 else None)
    ax.set_yticks(range(len(rows_lang)))
    ax.set_yticklabels([r[0] for r in reversed(rows_lang)])
    ax.set_xlim(14, 66)
    panel(ax, "first layer where Paris enters the top-5", "layer", axis="x")
    ax.legend(loc="lower right")
    plt.show()

for lang, p, fj, fl in rows_lang:
    gap = "-" if None in (fj, fl) else f"{fl - fj} layers earlier"
    print(f"{lang:9s} {p!r:34s} jacobian L{fj}    logit L{fl}    {gap}")
```

非英文的問句差距最大:Jacobian lens 很早就把英文的 Paris 排進前五,logit lens 要等到 50 幾層。方向跟 Anthropic 在 [On the Biology of a Large Language Model](https://transformer-circuits.pub/2025/attribution-graphs/biology.html) (<https://transformer-circuits.pub/2025/attribution-graphs/biology.html>) 講的多語言迴路一致,不過一個語言一條 prompt 證不了「不綁語言的概念空間」,只能說看到的 順序跟那個說法不衝突。

附錄 B:J-space 本體

主線讀出的都是「哪個 token 分數最高」。J-space 是比這更強的宣稱。

詞表裡每個 token t 在第 ℓ 層的 residual 裡對應一個方向

$$v_t = (W_U J_\ell)_t$$

也就是 $W_U J_\ell$ 的第 t 列。讀出時那個 token 的分數就是內積 $\langle v_t, h \rangle$, 前面每一次 readout 都是在算所有 token 的這個內積。

J-space 說的是 residual 的一小塊可以寫成少數幾個這種方向的非負組合:

$$h_\ell \approx \sum_{t \in S} a_t v_t, \quad a_t \geq 0, \quad |S| \leq 25$$

S 就是「此刻在 workspace 裡的東西」。非負是因為這些方向代表「有這個概念」,負係數 沒有對應的意思。

詞表有 248320 個方向,不可能全試,所以先用 readout 分數挑前 k 個候選,再解一個非負 最小平方。 k 掃 5、25、100,另外拿隨機挑的 25 個 token 當底線。

```
In [ ]: import functools

lm_head = llm.lm_head

def unembed_row(token):
    """One row of W_U, dequantized out of the 4-bit lm_head."""
    i = mx.array([token])
    return mx.dequantize(lm_head.weight[i], lm_head.scales[i], lm_head.biases[i],
                        group_size=lm_head.group_size, bits=lm_head.bits)[0]

@functools.lru_cache(maxsize=None)
def jlens_vec(token, l):
    """The J-lens direction of one token: row `token` of W_U @ J_l.

    Cached because appendices C and D ask for the same few tokens at every layer,
    and each call is a 5120x5120 matmul.
    """
    J = Jlens[l]
    return (unembed_row(token).astype(J.dtype) @ J).astype(mx.float32)

def nnls(V, target, steps=2000):
    """Smallest ||target - V a|| with every coefficient a >= 0.

    Projected gradient descent: a plain gradient step on the squared error, then
    clip negatives back to 0. Step 1/||G||_F is safe because the Frobenius norm
    is an upper bound on the largest eigenvalue of G, and gradient descent on a
    quadratic converges for any step below 2/lambda_max. A step picked by hand
    instead shows up as negative explained variance.
    """
    G = V.T @ V
    b = V.T @ target
    step = 1.0 / float(mx.sqrt(mx.sum(G * G)))
    a = mx.zeros((V.shape[1],))
    for _ in range(steps):
        a = mx.maximum(a - step * (G @ a - b), 0.0)
    return a

def jspace_fit(h, l, k=25, tokens=None):
    """Fit h with k J-lens directions. Returns (explained fraction, tokens, coefficients)."""
    if tokens is None:
        tokens = [int(t) for t in mx.argsort(-readout(transport(h, l)))[k:].tolist()]
    V = mx.stack([jlens_vec(t, l) for t in tokens], axis=1)
    a = nnls(V, h)
    left_over = h - V @ a
    explained = 1.0 - float(mx.sum(left_over ** 2) / mx.sum(h ** 2))
    return explained, a, tokens
```

```

In [ ]: KS = (5, 25, 100)
LAYERS = (48, 55, 60)
RANDOM_25 = [int(x) for x in mx.random.randint(0, 200000, (25,)),
              key=mx.random.key(0)).tolist()]

ids = encode(PROMPT)
res = residuals(ids)
last = ids.shape[1] - 1

explained, random_baseline, nonzero, best_fit = {}, {}, {}, None
for l in LAYERS:
    h = res[l][0, last].astype(mx.float32)
    for k in KS:
        e, toks, a = jspace_fit(h, l, k)
        explained[(l, k)] = e
        if k == 25:
            nonzero[l] = int(mx.sum(a > 1e-6))
            if l == LAYERS[-1]:
                best_fit = (toks, a)
            random_baseline[l] = jspace_fit(h, l, tokens=RANDOM_25)[0]

fig, axes = plt.subplots(1, 2, figsize=(7.4, 2.9))

ax = axes[0]
for l, color in zip(LAYERS, (LOGIT, JLENS, INK)):
    ax.plot(KS, [explained[(l, k)] * 100 for k in KS], marker="o", color=color,
            label=f"L{l}")
ax.plot(KS, [random_baseline[LAYERS[-1]] * 100] * len(KS), color=LOGIT, ls="--",
        lw=1.0, label="random 25 directions")
ax.set_xscale("log")
ax.set_xticks(KS)
ax.set_xticklabels([str(k) for k in KS])
panel(ax, "how much of the residual it accounts for", "k (directions used)",
      "explained %", 12)
ax.legend(loc="upper left")

toks, coeffs = best_fit
order = mx.argsort(-coeffs).tolist()
vals = [float(coeffs[i]) for i in order]
ax = axes[1]
ax.bar(range(len(vals)), vals, width=0.72,
      color=[JLENS if v > 1e-6 else "#dfe3e4" for v in vals])
panel(ax, f"L{LAYERS[-1]}: the 25 coefficients, sorted", "candidate", "coefficient")
plt.show()

for l in LAYERS:
    print(f"L{l} " + " ".join(f"k={k}: {explained[(l, k)] * 100:5.1f}%" for k in KS)
          + f" random-25: {random_baseline[l] * 100:4.1f}%"
          + f" nonzero coefficients {nonzero[l]}/25")
    print(f"\nL{LAYERS[-1]} candidates, by readout score: "
          + " ".join(repr(tokenizer.decode([t])) for t in toks[:6]))
    print(f"L{LAYERS[-1]} largest coefficients: "
          + " ".join(repr(tokenizer.decode([toks[i]])) for i in order[:6]))

```

25 個方向只解釋 residual 的百分之幾,加到 100 個也沒好多少,論文給的量級也是這樣。但隨機挑的 25 個方向差一到兩個數量級,所以挑出來的那 25 個是有結構的,不是隨便湊。

非負最小平方還會自己把一半的候選壓到 0,所以 $k = 25$ 是上限而不是實際用到的數目。

順著這裡也可以說「J-space」這個名字的問題:非負加上稀疏上限,這不是線性子空間,不能講「投影到 J-space」;照論文自己的 methods,它是一堆多面錐的聯集。

附錄 C:把 France 換成 China

對齊 [原文的實驗](https://www.anthropic.com/research/global-workspace) (<https://www.anthropic.com/research/global-workspace>):四個 prompt 分別問 France 的首都、語言、洲、貨幣,用完全相同的介入把 France 換成 China。四個答案一起改的話,最簡單的解釋是它們讀的是同一份表徵(論文說的 flexible generalization)。

做法是論文的 lens coordinate patching,用的就是附錄 B 那些 v_l :

$$V = [v_s \ v_t], \quad c = V^\dagger h, \quad h \leftarrow h + \alpha V(\sigma(c) - c)$$

c 是 h 在這兩個方向上的座標, σ 把兩個座標交換,跟 $\text{span}\{v_s, v_t\}$ 正交的成分不動。幾何上只動了這個平面,語義上是不是只換掉 France 這一個概念,公式不保證,要看輸出。

$V^\dagger$ 是 pseudo-inverse。不能直接拿內積當座標,因為這些方向彼此不正交(下面印出來的 cosine 就不是 0),直接取內積會把重疊的部分算兩次。

三個選擇決定成不成,都是掃出來的:

- 層 L40 到 L55。更早改幾乎沒反應,更晚改會把概念直接說出來(附錄 D.3 有數字)。
- 位置 只改 France 那個 token,不碰最後一個位置。最後幾層、最後一個位置的 J-lens 方向是「準備要說出口的字」,在那裡動手 model 會直接輸出 China。
- 強度 $\alpha = 1$ 是嚴格的交換, $\alpha = 2$ 是推過頭的版本,兩個都跑。

對照組 add only 只加上 China 的成分,不把 France 拿掉。

```
In [ ]: BAND = [l for l in Jlens if 40 <= l < 56]

def swap(x, v_src, v_tgt, alpha):
    """Exchange x's coordinates along v_src and v_tgt, leaving the rest of x alone."""
    V = mx.stack([v_src, v_tgt], axis=1)
    c = mx.linalg.pinv(V, stream=mx.cpu) @ x # pinv (not dot products): V's columns are not orthogonal
    return x + alpha * (V @ (c[:-1] - c)) # c[:-1] swaps the two coordinates

def add_only(x, v_src, v_tgt, alpha):
    """Control: add the target's component without removing the source's."""
    V = mx.stack([v_src, v_tgt], axis=1)
    c = mx.linalg.pinv(V, stream=mx.cpu) @ x
    return x + alpha * (c[0] - c[1]) * v_tgt

def edit_hook(edit, positions, layers=None):
    """A forward hook that rewrites the residual at `positions`, after every layer in `layers`.

    edit(x, layer) -> x takes and returns one float32 residual vector.
    """
    layers = BAND if layers is None else layers

    def hook(h, l):
        if l in layers:
            for p in positions:
                h[0, p] = edit(h[0, p].astype(mx.float32), l).astype(DTYPE)
            return h
    return hook

def concept_swap(src_word, tgt_word, alpha, kind=swap):
    return lambda x, l: kind(x, jlens_vec(tid(src_word), l),
                             jlens_vec(tid(tgt_word), l), alpha)

vf, vc = jlens_vec(tid(" France"), 48), jlens_vec(tid(" China"), 48)
print(f"v_France at L48: shape {tuple(vf.shape)}, norm {float(mx.linalg.norm(vf)):.1f}")
print(f"cos(v_France, v_China) = "
      f"{float(vf @ vc / (mx.linalg.norm(vf) * mx.linalg.norm(vc))):+.3f}")
print(f"patch band: L{BAND[0]}-{BAND[-1]} ({len(BAND)} layers)")
```

```
In [ ]: TASKS = [("capital", PROMPT, " Paris", " Beijing"),
               ("language", "The language spoken in France is", " French", " Chinese"),
               ("continent", "The continent that contains France is", " Europe", " Asia"),
               ("currency", "The currency of France is called the", " Euro", " Yuan")]

CONDS = [("before", None),
          ("swap a=1", concept_swap(" France", " China", 1.0)),
          ("swap a=2", concept_swap(" France", " China", 2.0)),
          ("add only a=2", concept_swap(" France", " China", 2.0, add_only))]

table = {}
for name, prompt, orig, new in TASKS:
    ids = encode(prompt)
    pos = where(ids, "France")
    for cond, edit in CONDS:
        hook = None if edit is None else edit_hook(edit, [pos])
        sc = final_scores(ids, hook)
        table[(name, cond)] = (prob(sc, orig), prob(sc, new), top1(sc))

fig, axes = plt.subplots(1, 4, figsize=(7.8, 2.6), sharey=True)
for ax, (name, _, orig, new) in zip(axes, TASKS):
    grouped_bars(ax, ["before", "a=1", "a=2", "add\nonly"],
                  {"original answer": [table[(name, c)][0] for c, _ in CONDS],
                  "swapped in": [table[(name, c)][1] for c, _ in CONDS]},
                  [LOGIT, SWAPPED])
    panel(ax, f"{name}\n{orig.strip()} -> {new.strip()}", "", "", 0.72)
axes[0].set_ylabel("probability")
axes[-1].legend(loc="upper right")
plt.show()

for name, prompt, orig, new in TASKS:
    print(f"{prompt!r}")
    for cond, _ in CONDS:
        po, pn, t = table[(name, cond)]
        print(f"    {cond:13s} top1={t!r:11s} p({orig.strip()}={po:.3f}  p({new.strip()}={pn:.3f})")
```

一個介入、四個問題,看四個答案是不是一起改口(Paris→Beijing、French→Chinese、Europe→Asia、Euro→Yuan)。完整版跑出來四個都要 $\alpha = 2$ 才翻得過去, $\alpha = 1$ 方向對但推不過去,而對照組 add only 四個都沒翻——也就是「把 France 拿掉」這一半是必要的。

幅度要留意。Neel Nanda 團隊在同一類 model 上複現(<https://www-cdn.anthropic.com/files/4zrzovbb/website/cc4be2488d65e54a6ed06492f8968398ddc18ebe.pdf>) 時也只拿到微弱但為正的因果效果,這是一顆 27B 的 4-bit 量化 model,不是 Sonnet 4.5。

附錄 D:三個 workspace 面向

論文給了五個功能面向,要一項一項成立才算 workspace。附錄 C 做的是 flexible generalization,這裡再抽三個能用同一套機制跑的。

D.1 定向調節:叫它想,叫它不要想

同一句抄寫任務,後面接不同的指令,量 J-space 裡有沒有那個概念。兩組設計:

- 白熊: bear 在兩個 prompt 裡都出現,「因為輸入有這個字才讀到」的效應互相抵銷,差的只有指令。
- lemon:指令說的是 citrus fruit, lemon 從頭到尾沒出現在 prompt 裡,讀到它就不能用 input-copying 解釋。對照組把指令換成想呼吸,測「任何指令都會抬高一切」。

讀法是掃 L40 到 L60、每一層每一個位置,取最大的機率。

```

In [ ]: SCAN = [l for l in Jlens if 40 <= l < 62 and l % 2 == 0]

def max_jlens_prob(ids, words, layers=SCAN):
    """Largest J-lens probability of any spelling in `words`, over all layers x positions."""
    res = residuals(ids)
    best = (0.0, None, None)
    for l in layers:
        p = mx.softmax(readout(transport(res[l], l)), axis=-1)[0] # (position, vocab)
        for w in words:
            column = p[:, tid(w)]
            if float(mx.max(column)) > best[0]:
                best = (float(mx.max(column)), l, int(mx.argmax(column)))
    return best

TASK = 'Write the sentence "The old painting hung crookedly on the wall."'

CASES = [
    ("bear (in the prompt)", (" bear", " bears"), [
        ("not mentioned", TASK),
        ("think about it", TASK + " While you write, think about a white bear."),
        ("do NOT think of it", TASK + " While you write, do not think about a white bear.")] ),
    ("lemon (never in the prompt)", (" lemon", " lemons"), [
        ("no instruction", TASK),
        ("think about citrus", TASK + " While you write, think about citrus fruit."),
        ("think of breathing", TASK + " While you write, think about your breathing.")] ),
]

found = [(target, tag) + max_jlens_prob(encode(prompt), words)
         for target, words, cases in CASES for tag, prompt in cases]

fig, axes = plt.subplots(1, 2, figsize=(7.2, 2.9), sharey=True)
for ax, (target, _, cases) in zip(axes, CASES):
    vals = [f[2] for f in found if f[0] == target]
    ax.bar(range(len(vals)), vals, width=0.62, color=[LOGIT, JLENS, SWAPPED])
    for i, v in enumerate(vals):
        ax.text(i, max(v, 6e-5) * 1.3, f"{v:.4f}", ha="center", fontsize=7.5,
                color=INK) # max(): a 0 would fall off the log axis
    ax.set_xticks(range(len(cases)))
    ax.set_xticklabels([tag.replace(" ", "\n") for tag, _ in cases], fontsize=7.5)
    ax.set_yscale("log") # the values span 3 orders of magnitude
    ax.set_ylim(5e-5, 1.0)
    panel(ax, target)
axes[0].set_ylabel("max J-lens p over layers x positions")
plt.show()

for target, tag, p, l, pos in found:
    print(f"{target:28s} {tag:20s} max p = {p:.4f} at L{l} position {pos}")

```

絕對值都很小,要看的是同一個 target 在不同指令之間的比值。完整版的結果:白熊那組成立,而且方向是反的——叫它不要想白熊,白熊在 J-space 裡比叫它想的時候還清楚一個數量級(論文圖 10 有跨 model 的版本)。lemon 那組是乾淨的正向,換成想呼吸就掉回基準,所以動的是被指定的那個概念,不是有指令就整體抬高。

D.2 把概念消除

交換是把座標換過去,消除是把座標歸零,同一套機制取 $\sigma(c) = 0$:

$$V = [v_1 \cdots v_k], \quad h \leftarrow h - \alpha V(V^\dagger h)$$

一次投掉整組方向。這些方向彼此不正交,一個一個減會重複扣掉共用的成分。介入從最小的開始加碼:一個方向一個位置,到一個方向所有位置,再到整組 {France, French, Paris}。

```
In [ ]: def ablate(words, alpha=1.0):
        """ $h \leftarrow h - \alpha V$  ( $V+h$ ): project the span of several J-lens directions out of  $h$ ."""
        def edit(x, l):
            V = mx.stack([jlens_vec(tid(w), l) for w in words], axis=1)
            return x - alpha * (V @ (mx.linalg.pinv(V, stream=mx.cpu) @ x))
        return edit

ids = encode(PROMPT)
france_pos = where(ids, "France")
every_pos = list(range(ids.shape[1]))

RUNS = [
    ("baseline", None),
    ("-France @ France position", edit_hook(ablate([" France"]), [france_pos])),
    ("-France @ every position", edit_hook(ablate([" France"]), every_pos)),
    ("-{France, French, Paris} @ every position",
     edit_hook(ablate([" France", " French", " Paris"]), every_pos)),
]

out = []
for tag, hook in RUNS:
    sc = final_scores(ids, hook)
    out.append((tag, prob(sc, " Paris"), top1(sc)))

fig, ax = plt.subplots(figsize=(6.2, 2.4))
ys = list(range(len(out))[:-1])
ax.barh(ys, [o[1] for o in out], height=0.66,
        color=[LOGIT if o[2] == " Paris" else SWAPPED for o in out])
for y, (tag, v, t) in zip(ys, out):
    ax.text(v + 0.012, y, f"{v:.3f}" + (" if t == " Paris" else f" top-1 {t!r}"),
            va="center", fontsize=7.5, color=INK)
ax.set_yticks(ys)
ax.set_yticklabels([o[0] for o in out], fontsize=7.5)
ax.set_xlim(0, 0.9)
panel(ax, f"p(Paris) for {PROMPT!r}", "p(Paris)", axis="x")
plt.show()

for tag, v, t in out:
    print(f"{tag:42s} p(Paris)={v:.3f} top1={t!r}")
```

消除成不成看你動多少。完整版的數字:只投掉 France 那個位置的一個方向, $p(\text{Paris})$ 從 0.617 掉到 0.524;同一個方向每個位置都投,掉到 0.197;整組 {France, French, Paris} 每個位置都投,0.001,top-1 換人。

概念不是只存在提到它的那個位置上, France 的資訊在 forward 的過程中已經散到別的位置, 只掐一個點沒用。

D.3 內部推理:交換之後,它會拿新概念去算嗎

這一項要求交換之後 model 用新概念做推論,而不是把它輸出。題目是兩跳事實:

The capital of the country where the Eiffel Tower stands is -> Paris

中間那一跳 France 完全不在 prompt 裡,是 model 自己算出來的。把 France 換成 China 之後,答 Beijing 代表它拿新概念又跑了一次第二跳,答 China 代表它只是把注入的東西講出來。這個題型能把推理和洩漏分開。

France 不在輸入裡,所以沒有「France 那個 token 的位置」可以改,只能改別的位置,三個選擇都試。

```
In [ ]: TWO_HOP = "The capital of the country where the Eiffel Tower stands is"
ids = encode(TWO_HOP)
last = ids.shape[1] - 1

PLACES = [("last position only", [last]),
          ("Tower position only", [where(ids, "Tower")]),
          ("every position but the last", list(range(1, last)))]
WATCH = [("Paris", " Paris"), ("Beijing", " Beijing"), ("France", " France")]
ALPHAS = (0, 1.0, 2.0)

hop = {}
for tag, positions in PLACES:
    for alpha in ALPHAS:
        hook = None if alpha == 0 else edit_hook(
            concept_swap(" France", " China", alpha), positions)
        sc = final_scores(ids, hook)
        hop[(tag, alpha)] = ({name: prob(sc, w) for name, w in WATCH}, top1(sc))

fig, axes = plt.subplots(1, 3, figsize=(7.8, 2.6), sharey=True)
for ax, (tag, _) in zip(axes, PLACES):
    for (name, _) in zip(WATCH, (LOGIT, SWAPPED, INK)):
        ax.plot(ALPHAS, [hop[(tag, a)][0][name] for a in ALPHAS], marker="o",
                color=color, label=f"p({name})")
    panel(ax, f"patch {tag}", "alpha", "", 1.05)
axes[0].set_ylabel("probability")
axes[2].legend(loc="upper left")
plt.show()

print(f"{TWO_HOP!r} ('France' in the prompt: {'France' in TWO_HOP})")
for tag, positions in PLACES:
    print(f" patch {tag} {positions}")
    for alpha in ALPHAS:
        watched, t = hop[(tag, alpha)]
        cols = " ".join(f"p({n})={watched[n]:.3f}" for n, _ in WATCH)
        print(f" alpha={alpha:<4} {cols} top1={t!r}")
```

三個位置選擇的完整版結果:只改最後一個位置, $\alpha = 2$ 時 $p(\text{France})$ 衝到 0.975, top-1 就是 ' France' —— 它把被交換的概念直接說出來,這就是附錄 C 提的 late-layer 洩漏; 只改 Tower 那個位置幾乎沒反應,單一個位置扛不起來;除了最後一個以外全改,top-1 變成 ' Beijing' ,而 $p(\text{France})$ 、 $p(\text{China})$ 都還在 0.001 以下。

最後那個是要找的結果:它沒有把 China 講出來,講的是第二跳的結果,而被換掉的概念從頭到尾不在輸入裡,所以也不可能是 input-copying。

五個面向跑完的樣子

完整版把論文五個面向加上概念消除全部跑過,結論是:

面向	這類 model(Qwen3.6-27B-4bit)	在哪
flexible generalization	成立。一個介入,四個任務一起改	附錄 C
directed modulation	成立。叫它不要想白熊,白熊反而更清楚	D.1
internal reasoning	成立,但只有兩跳事實那種,數腳那種不行	D.3
概念消除(論文驗因果的手法)	成立,但要投整組方向、每個位置都投	D.2
selectivity	一半。明確報告翻得動,但拿不到跟自動處理的乾淨分離	完整版 §7.3
verbal report	不成立。注入之後,問什麼都會說出那個字	完整版 §7.4

成立的那幾項有一個共同條件:介入要落在內容位置,而且往往要落在一整片位置上。只改一個點 不夠,改最後一個位置會退化成把概念直接說出來。